Version Control Using Git

Title

Version Control Using Git

Objective

The objective of this activity was to understand the fundamentals of Version Control Systems (VCS), learn the features and architecture of Git, perform common Git operations, and understand how Git helps developers collaborate efficiently while maintaining project history.

Introduction

Software development involves continuous modifications to source code. Without a proper system, managing different versions of files becomes difficult, especially when multiple developers work on the same project. A **Version Control System (VCS)** is a tool that records changes made to files over time so that developers can review, restore, or compare previous versions whenever required.

Git is the most widely used distributed Version Control System. It helps developers track code changes, collaborate with team members, create branches for new features, merge completed work, and maintain a complete history of the project.

Git is fast, secure, open-source, and supports both individual and team-based software development.

What is Version Control?

Version Control is the process of tracking and managing changes to files and source code. Every modification is recorded, allowing developers to restore earlier versions if needed.

A Version Control System helps to:

- Track changes in source code.
- Restore previous versions.
- Prevent accidental data loss.
- Enable multiple developers to work simultaneously.
- Maintain project history.

- Improve software quality.

Types of Version Control Systems

1. Local Version Control System

Stores file versions only on the local computer.

Advantages

- Simple to use.
- Fast for individual work.

Disadvantages

- No collaboration.
- Risk of losing data if the system fails.

2. Centralized Version Control System (CVCS)

A single central server stores the complete project repository.

Examples

- SVN (Subversion)
- CVS

Advantages

- Easy collaboration.
- Centralized project management.

Disadvantages

- If the server fails, development is interrupted.

3. Distributed Version Control System (DVCS)

Each developer has a complete copy of the repository, including its history.

Examples

- Git
- Mercurial

Advantages

- Works offline.
- Faster operations.
- Better collaboration.
- Complete backup on every developer's machine.

What is Git?

Git is a Distributed Version Control System created by **Linus Torvalds** in **2005** for Linux kernel development.

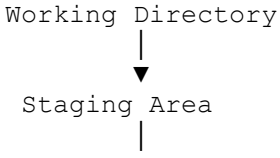
Git allows developers to:

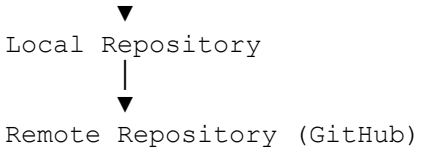
- Track file changes.
- Create branches.
- Merge different versions.
- Collaborate using GitHub, GitLab, or Bitbucket.
- Roll back to previous versions.

Features of Git

- Open Source
- Distributed Architecture
- High Performance
- Lightweight
- Secure Data Storage
- Branching and Merging
- Complete Version History
- Easy Collaboration
- Supports Large Projects
- Fast Operations

Git Architecture





Explanation

Working Directory

The location where developers create and edit project files.

Staging Area

Temporary area where selected changes are prepared before committing.

Local Repository

Stores all committed versions on the local computer.

Remote Repository

Stores the project online for collaboration and backup.

Basic Git Workflow

1. Create or modify files.
2. Check repository status.
3. Add files to the staging area.
4. Commit changes.
5. Push changes to GitHub.
6. Pull updates from the remote repository.

Common Git Commands

Initialize Repository

```
git init
```

Creates a new Git repository.

Check Status

```
git status
```

Displays modified, staged, and untracked files.

Add Files

```
git add .
```

Adds all files to the staging area.

Commit Changes

```
git commit -m "Initial Commit"
```

Creates a snapshot of the project.

View Commit History

```
git log
```

Displays previous commits.

Create Branch

```
git branch feature
```

Creates a new branch.

Switch Branch

```
git checkout feature
```

Moves to another branch.

Merge Branch

```
git merge feature
```

Combines another branch into the current branch.

Connect GitHub Repository

```
git remote add origin https://github.com/username/project.git
```

Connects the local repository to GitHub.

Push Files

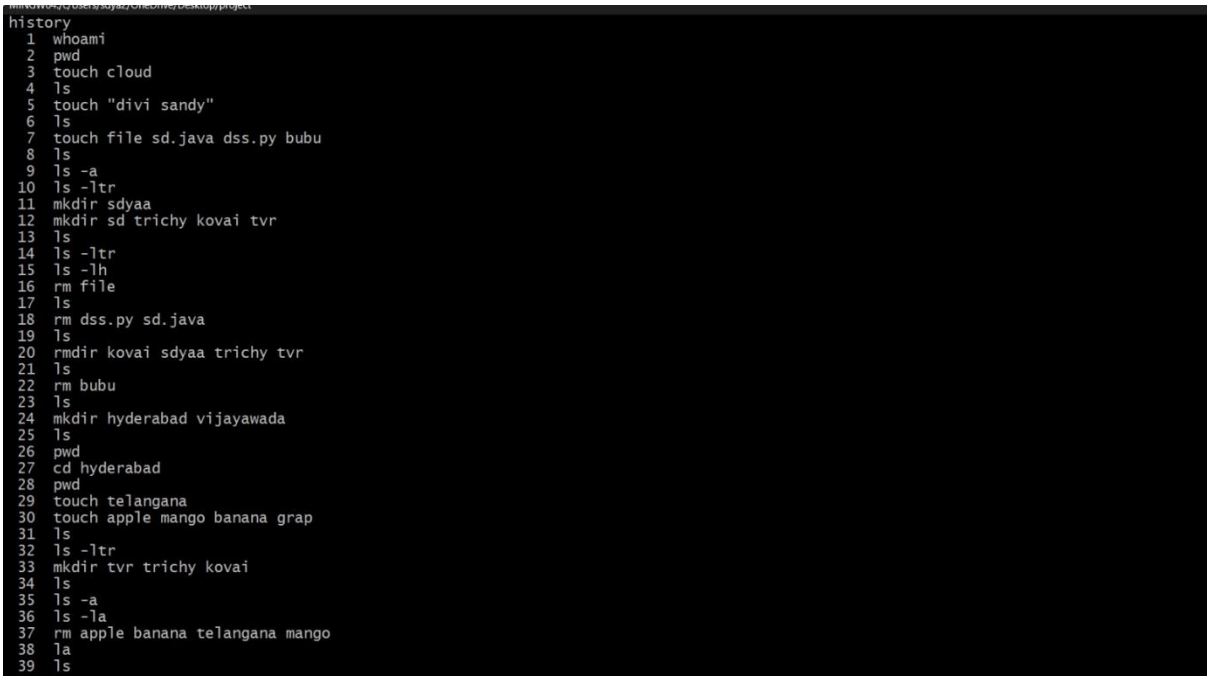
```
git push -u origin main
```

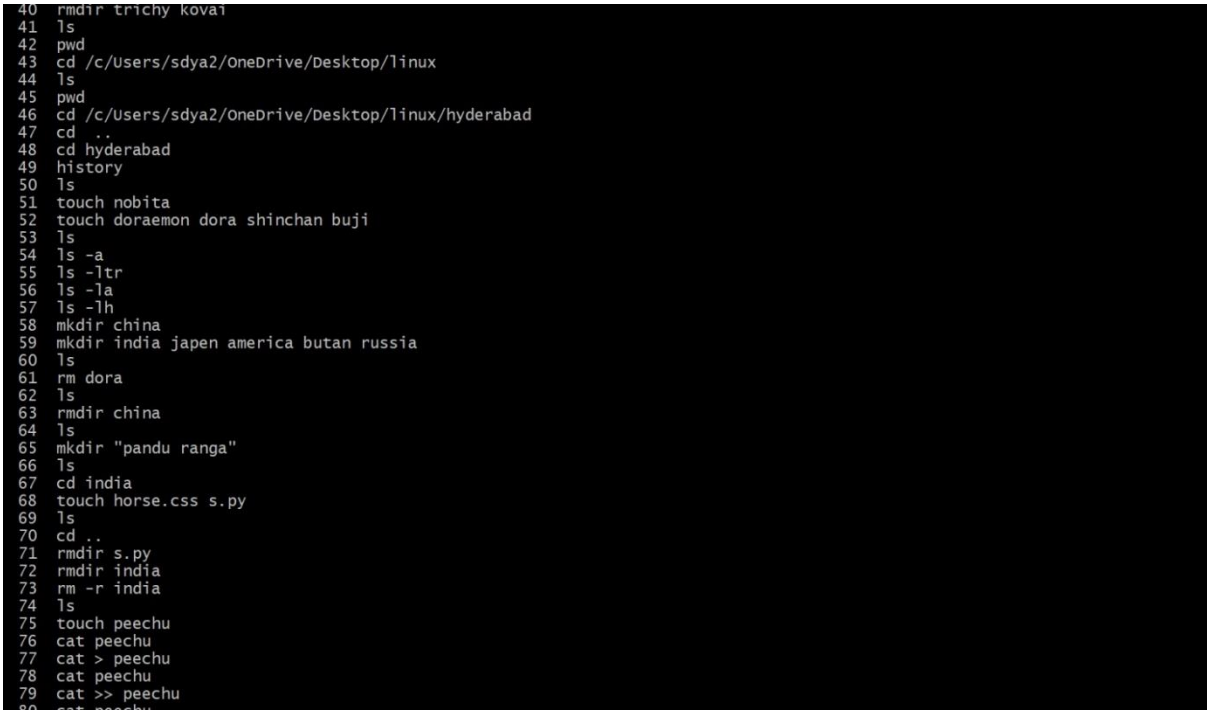
Uploads commits to GitHub.

Pull Updates

```
git pull origin main
```

Downloads the latest changes from GitHub.





```
40  rmdir trichy kovai
41  ls
42  pwd
43  cd /c/Users/sdya2/OneDrive/Desktop/linux
44  ls
45  pwd
46  cd /c/Users/sdya2/OneDrive/Desktop/linux/hyderabad
47  cd  ..
48  cd hyderabad
49  history
50  ls
51  touch nobita
52  touch doraemon dora shinchan buji
53  ls
54  ls -a
55  ls -ltr
56  ls -la
57  ls -lh
58  mkdir china
59  mkdir india japen america butan russia
60  ls
61  rm dora
62  ls
63  rmdir china
64  ls
65  mkdir "pandu ranga"
66  ls
67  cd india
68  touch horse.css s.py
69  ls
70  cd ..
71  rmdir s.py
72  rmdir india
73  rm -r india
74  ls
75  touch peechu
76  cat peechu
77  cat > peechu
78  cat peechu
79  cat >> peechu
```

```
 78  cat peechu
 79  cat >> peechu
 80  cat peechu
 81  cat > peechu
 82  cat peechu
 83  vi peechu
 84  git version
 85  touch divi.html
 86  cat > divi.html
 87  cat divi.html
 88  git status
 89  git init
 90  ls
 91  git status
 92  git add divi.html
 93  git status
 94  touch app.js sandy.py kani.css pandu.js csea
 95  git add .
 96  git status
 97  touch cse it aiml datascience
 98  git status
 99  git add .
100  git config --global use.email "sdya2005@gamil.com"
101  git config --global user.name"dhivya"
102  git config --global use.email "sdya2005@gamil.com"
103  git config --global user.email "sdya2005@gmail.com"
104   git config --global user.name"dhivya"
105  git commit -m "my first commit"
106  git log
107  touch Dockerfile
108  git status
109  git add Dockerfile
110  git status
111  git commit -m "added new file called Dockerfile"
112  cat > Dockerfile
113  git status
114  git add Dockerfile
115  git status
116  git restore --staged Dockerfile
117  git status
118  git add Dockerfile
```

Applications of Git

- Software Development
- Web Development
- Mobile App Development
- DevOps

- Cloud Projects
- Open Source Projects
- Team Collaboration
- Continuous Integration (CI/CD)

Advantages of Git

- Tracks every project change.
- Supports collaboration.
- Maintains complete project history.
- Allows rollback to previous versions.
- Supports branching and merging.
- Fast and efficient.
- Free and open source.
- Provides secure backup through GitHub.

Limitations of Git

- Learning curve for beginners.
- Merge conflicts can occur.
- Large binary files are difficult to manage.
- Requires proper branch management.

Learning Outcome

During this activity, I learned:

- The concept of Version Control Systems.
- Differences between Local, Centralized, and Distributed Version Control.
- Git architecture and workflow.
- Basic Git commands.
- How to connect a local repository with GitHub.
- How Git improves collaboration and project management.

Conclusion

The Version Control using Git activity was successfully completed. Git provides an efficient and reliable method for tracking changes, managing source code, and collaborating with development teams. Its distributed architecture, branching capabilities, and integration with platforms like GitHub make it an essential tool for modern software development. This practical session enhanced my understanding of version control concepts and strengthened my skills in using Git for real-world projects.